

SPECRESEARCH LOOP

Mục tiêu

Đồ án cuối kỳ:

https://docs.google.com/document/d/1L7rkfkAKHedJyfnMLUatxM_WW-LS0A7B/edit?usp=sharing&oid=103209990818711181497&rtpof=true&sd=true

Qua bài tập này, sinh viên cần hiểu:

- Hệ thống **SpecResearch Loop** thực sự phải làm gì.
- Những vấn đề kỹ thuật sẽ gặp khi triển khai.
- Có thể dùng giải pháp kỹ thuật nào để xử lý.
- Các thành phần phải phối hợp với nhau như thế nào.

SpecResearch Loop không chỉ là chatbot hỏi–đáp. Hệ thống phải liên tục xây dựng và cập nhật:

Idea

- Problem
- Gap
- Contribution
- Claim
- Evidence
- Experiment
- Judge
- User sửa
- Spec mới
- Judge lại
- Final Spec

Tài liệu yêu cầu hệ thống quản lý nguồn, experiment plan, nhiều Judge, quyết định của người dùng, version và diff của research spec.

PHẦN A – HIỂU HỆ THỐNG

Câu 1 – Hệ thống đang xây dựng cái gì?

Người dùng nhập:

“Tôi muốn dùng LLM để giảm hallucination khi đọc paper.”

Sau một số vòng, hệ thống có:

Problem:

LLM có thể sinh thông tin không được paper hỗ trợ.

Gap:

Chưa rõ claim-level evidence feedback có cải thiện vấn đề không.

Contribution:

Dùng verifier kiểm tra từng claim.

Claim:

Phương pháp giảm unsupported claims.

Experiment:

So sánh với Self-Refine và OPRO.

Câu hỏi

1. Theo bạn, hệ thống đang tạo **một đoạn văn**, hay đang xây dựng **một Research Spec có cấu trúc**?
2. Những thành phần trên liên quan với nhau như thế nào?
3. Nếu Gap thay đổi thì Contribution, Claim, Experiment có cần xem lại không? Vì sao?

Gợi ý minh họa

Gap



Contribution



Claim



Experiment

Nếu sửa phần trên, phần dưới có thể không còn đúng.

Câu 2 – Requirement nào tạo ra vấn đề kỹ thuật?

Ví dụ requirement:

“Mỗi claim phải có evidence.”

Nhìn bên ngoài rất đơn giản.

Nhưng khi code sẽ xuất hiện câu hỏi:

Làm sao biết paper thực sự hỗ trợ claim?

Nhiệm vụ

Với **3 requirement** dưới đây, hãy viết:

Requirement

→ Technical Problem

→ Giải pháp có thể dùng

Requirement A

Hệ thống phải nhớ các quyết định trước đó của người dùng.

Requirement B

Citation phải thực sự hỗ trợ claim.

Requirement C

Khi người dùng sửa Research Gap, hệ thống phải cập nhật các phần liên quan.

Ví dụ:

Requirement:

Nhớ quyết định của người dùng

Technical Problem:

LLM call mới không tự biết toàn bộ trạng thái trước đó.

Possible Solution:

Lưu Research Spec + Decision History vào database.

PHẦN B – CÁC VẤN ĐỀ KỸ THUẬT QUAN TRỌNG

Câu 3 – Làm sao giữ context xuyên suốt nhiều vòng?

Đây là một vấn đề quan trọng của đề án.

Giả sử hệ thống chạy:

Loop 1:

User Idea

↓

Spec v1

Loop 2:

User làm rõ Problem

↓

Spec v2

Loop 3:
Tìm Related Work
↓
Spec v3

Loop 4:
User sửa Gap
↓
Spec v4

Loop 5:
Judge đánh giá
↓
?

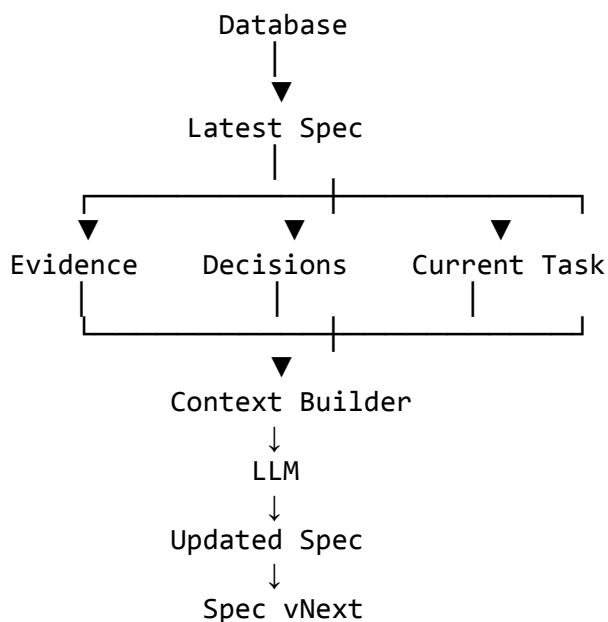
Vấn đề

Ở Loop 5, Judge phải biết **Spec v4**, không được vô tình đánh giá Spec v2 hoặc Spec v3.

Câu hỏi

1. Hệ thống nên lấy “context hiện tại” từ đâu?
2. Có nên gửi toàn bộ lịch sử chat từ đầu cho LLM mỗi lần không?
3. Làm sao biết đâu là **latest spec**?
4. Khi mỗi bước cập nhật spec, nên lưu như thế nào?

Một hướng thiết kế để tham khảo



Ví dụ:

Spec v3
Gap = A

User:
"Tôi muốn đổi Gap thành B"

→ tạo Spec v4

Judge chạy sau đó phải nhận:
Spec v4
không phải Spec v3.

Yêu cầu

Hãy đề xuất cách nhóm bạn quản lý:

- latest_spec;
 - spec_version;
 - decision_history;
 - evidence;
 - context gửi cho LLM.
-

Câu 4 – Có nên gửi toàn bộ dữ liệu vào prompt?

Sau 10 vòng, hệ thống có:

- 50 paper;
- 200 đoạn evidence;
- 20 quyết định của user;
- 5 version của spec;
- nhiều kết quả Judge.

Nếu đưa tất cả vào prompt:

Everything



LLM

có thể rất dài, tốn token và làm LLM khó tập trung.

Câu hỏi

Hãy đề xuất cách tạo **context** đúng cho từng task.

Ví dụ:

Gap Judge cần:

Current Gap

+ Related Work

+ Relevant Evidence

không nhất thiết cần toàn bộ UI history.

Experiment Judge cần:

Current Claims

+ Experiment Plan

+ Baselines

+ Metrics

Nhiệm vụ

Hoàn thành:

Task

→ Context cần thiết

cho:

1. Gap Judge.
2. Evidence Judge.
3. Experiment Judge.

Câu 5 – Paper liên quan chưa chắc là Evidence

Hệ thống có claim:

“TextGrad không sử dụng claim-level evidence verification.”

Search tìm được TextGrad với similarity:

0.92

Câu hỏi

Similarity cao có đủ để kết luận paper hỗ trợ claim không?

Hãy thiết kế pipeline đơn giản:

Claim

↓

Search paper

↓

Tìm đoạn liên quan

↓

?

↓
Supported / Contradicted / Insufficient

Sinh viên có thể cân nhắc:

- Embedding Search
- Reranker
- LLM Verifier
- NLI
- Rule
- Human Check

Điều quan trọng là phân biệt:

Relevant Paper

≠

Supporting Evidence

Tài liệu đồ án cũng yêu cầu mỗi nhận định phải liên kết với nguồn cụ thể và cảnh báo khi nguồn không trực tiếp hỗ trợ claim.

Câu 6 – Không tìm thấy paper có phải Research Gap?

Hệ thống tìm 30 paper nhưng không thấy paper nào giống ý tưởng.

LLM kết luận:

“Đây là research gap mới.”

Câu hỏi

Điều này có hợp lý không?

Giải thích sự khác biệt:

Không tìm thấy paper

≠

Paper không tồn tại

≠

Đây chắc chắn là Research Gap

Hãy đề xuất **2–3 cách** giúp kiểm tra gap tốt hơn.

Ví dụ:

Query 1

↓

Query expansion

↓

Search bằng từ đồng nghĩa



Citation / related-paper search



Counter-evidence search



Gap Candidate

Đồ án nêu trực tiếp rằng research gap không được tạo đơn giản theo kiểu “tôi chưa thấy paper giống hệt nên đây là gap”.

Câu 7 – Nhiều Judge nhưng Judge bất đồng

Kết quả:

Gap Judge	ACCEPT
Contribution Judge	ACCEPT
Evidence Judge	REJECT
Experiment Judge	MAJOR
Conference Judge	7/10

Câu hỏi

Hệ thống nên làm gì?

Chọn hoặc đề xuất cách khác:

- A. Majority vote
- B. Judge nào cũng ngang nhau
- C. Một số lỗi Critical có quyền block
- D. Đưa disagreement cho người dùng quyết định

Ví dụ:

Nếu:

Evidence Judge = REJECT

vì citation không hỗ trợ claim,

thì dù các Judge còn lại đồng ý, có nên cho spec qua không?

Giải thích.

PHẦN C – KHI HỆ THỐNG CHẠY THỰC TẾ

Câu 8 – User sửa một phần, chạy lại tất cả?

Hiện tại:

Problem ✓
Gap ✓
Contribution ✓
Claim ✓
Experiment ✓
Judge ✓

User sửa:

Gap v3 → Gap v4

Câu hỏi

Có cần chạy lại từ đầu không?

Hãy đánh dấu:

Problem	KEEP / UPDATE?
Related Work	KEEP / UPDATE?
Gap	UPDATE
Contribution	?
Claim	?
Experiment	?
Judge	?

Gợi ý

Có thể xây dependency:

```
Gap
↓
Contribution
↓
Claim
↓
Experiment
↓
Judge
```

Từ đó:

```
Change
↓
Find affected components
↓
```

Invalidate
↓
Recompute
↓
Judge again

Câu 9 – Hệ thống crash giữa vòng

Workflow chạy 15 phút:

1 Idea	✓
2 Clarification	✓
3 Research	✓
4 Gap	✓
5 Contribution	✓
6 Experiment	✓
7 Judge	← API timeout

Câu hỏi

Có nên quay lại Step 1?

Nếu không, cần kỹ thuật gì?

Có thể cân nhắc:

Checkpoint
Persistent State
Retry
Caching
Fallback Model
Versioning

Ví dụ:

Database:
current_step = 7
latest_spec = v6

API crash

→ Retry Step 7

thay vì:

→ chạy lại từ Step 1.

Câu 10 – Làm sao biết hệ thống thật sự tốt?

Nhóm tuyên bố:

“Multi-Judge tốt hơn Single Judge.”

Không được chỉ demo vài ví dụ rồi kết luận.

Câu hỏi

Hãy thiết kế một thí nghiệm đơn giản:

Baseline:

?

Dataset:

?

Ground Truth:

?

Metric:

?

Proposed:

?

Ví dụ:

100 Research Specs

↓

Giảng viên đánh dấu lỗi thật

↓

Single Judge

vs

Multi-Judge

↓

So sánh:

Precision

Recall

Cost

Latency

Đồ án yêu cầu mỗi cải tiến phải trả lời được ba vấn đề: **giải quyết vấn đề gì, kiểm nghiệm như thế nào và có tốt hơn baseline hay không.**

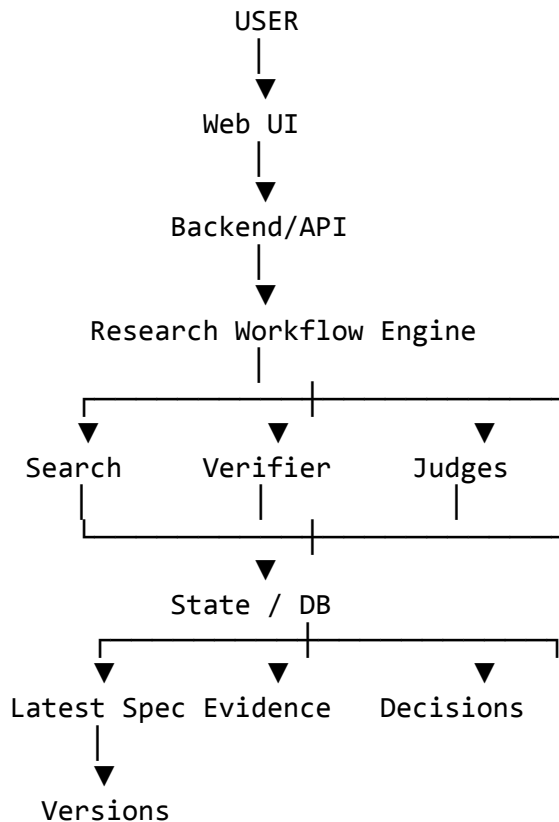
PHẦN D – VẼ KIẾN TRÚC

Câu 11 – Ghép tất cả thành một hệ thống

Hãy vẽ kiến trúc SpecResearch Loop.

Không cần quá chi tiết.

Có thể bắt đầu từ:



Phải trả lời được 5 câu

1. Latest Spec nằm ở đâu?
2. Ai tạo context cho mỗi LLM call?
3. Khi user sửa một phần, ai quyết định phần nào phải chạy lại?
4. Evidence được lưu và liên kết với Claim như thế nào?
5. Nếu UI thay bằng mobile hoặc CLI, workflow có phải viết lại không?

CÂU HỎI XUYỀN SUỐT BÀI

Khi gặp bất kỳ requirement nào, hãy tự hỏi:

Requirement nói gì?



Khi code sẽ gặp vấn đề gì?



Giải pháp kỹ thuật nào?



Giải pháp có thể sai ở đâu?



Làm sao biết nó hoạt động?

Ví dụ:

Requirement:

"Giữ context xuyên suốt nhiều loop"



Problem:

LLM call mới không tự biết trạng thái mới nhất.



Solution:

State Store

+ Versioned Research Spec

+ Context Builder

+ Decision History



Failure:

Judge dùng nhầm Spec cũ.



Verification:

Mỗi call lưu:

spec_version_used

input_context

output_version

Đó chính là kiểu tư duy cần dùng khi thực hiện **SpecResearch Loop**.